

SIMATS ENGINEERING

ASSESSMENT TOOL 2 (Medium)- Debugging and Optimization

COURSE NAME: Data Structures

COURSE CODE: CSA0316

NAME: Sarah Sharmeen. R

REG NO: 192571073

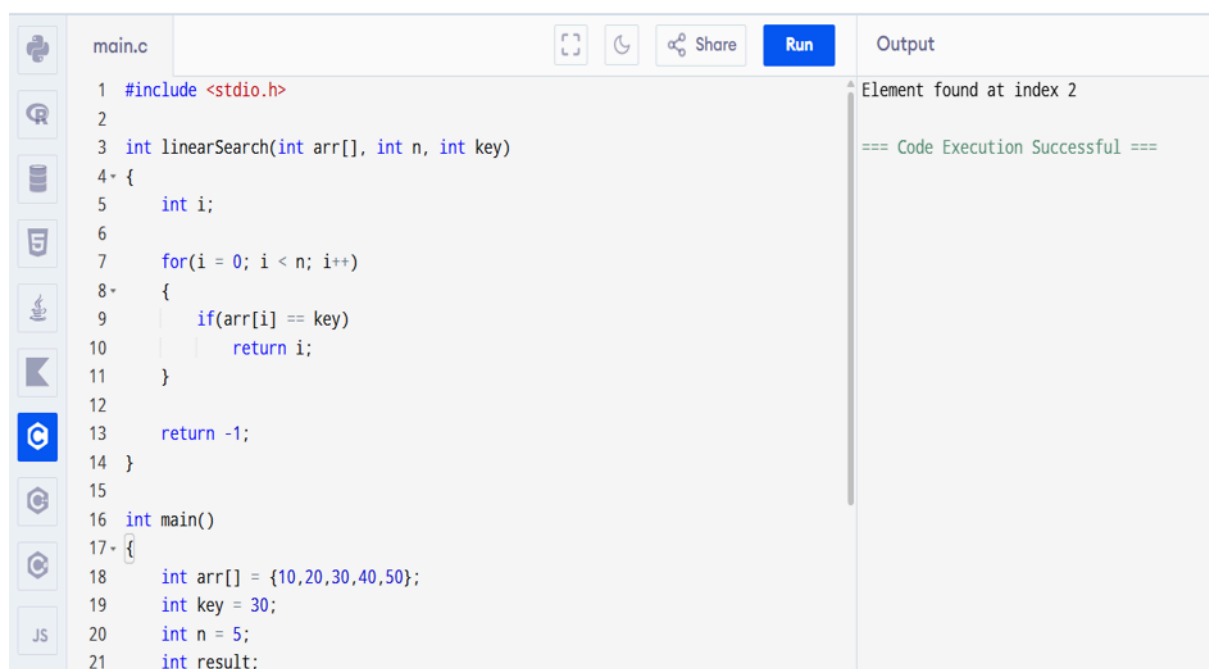
COURSE OUTCOME COVERED

CO2 : Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques. (BTL3, BTL4)

Assessment Tool Weightage: **10%**

1. Find the bugs, include the missing lines in the following snippet.

```
int linearSearch(int arr[], int n, int key) {  
  
for(int i = 0; i <= n; i++) {  
  
if(arr[i] == key)  
  
return i;  
  
}  
  
return -1;
```



The screenshot shows a C++ IDE with a file named 'main.c'. The code defines a linear search function and a main function. The function 'linearSearch' takes an array 'arr', its size 'n', and a 'key' as input. It iterates from 'i = 0' to 'i < n' (note: the code has a bug here, it should be 'i < n' instead of 'i <= n'). If it finds the key, it returns the index 'i'. If it doesn't find the key, it returns -1. The main function initializes an array 'arr' with values {10, 20, 30, 40, 50}, sets 'key' to 30, 'n' to 5, and declares a variable 'result'. The output window shows 'Element found at index 2' and '=== Code Execution Successful ==='.

```
1 #include <stdio.h>  
2  
3 int linearSearch(int arr[], int n, int key)  
4 {  
5     int i;  
6  
7     for(i = 0; i < n; i++)  
8     {  
9         if(arr[i] == key)  
10            return i;  
11     }  
12  
13     return -1;  
14 }  
15  
16 int main()  
17 {  
18     int arr[] = {10,20,30,40,50};  
19     int key = 30;  
20     int n = 5;  
21     int result;
```

Output
Element found at index 2
=== Code Execution Successful ===

2. Buggy Code in an array

```
#include <stdio.h>

int main() {

    int arr[5] = {10,20,30,40,50};

    for(int i=0;i<=5;i++)

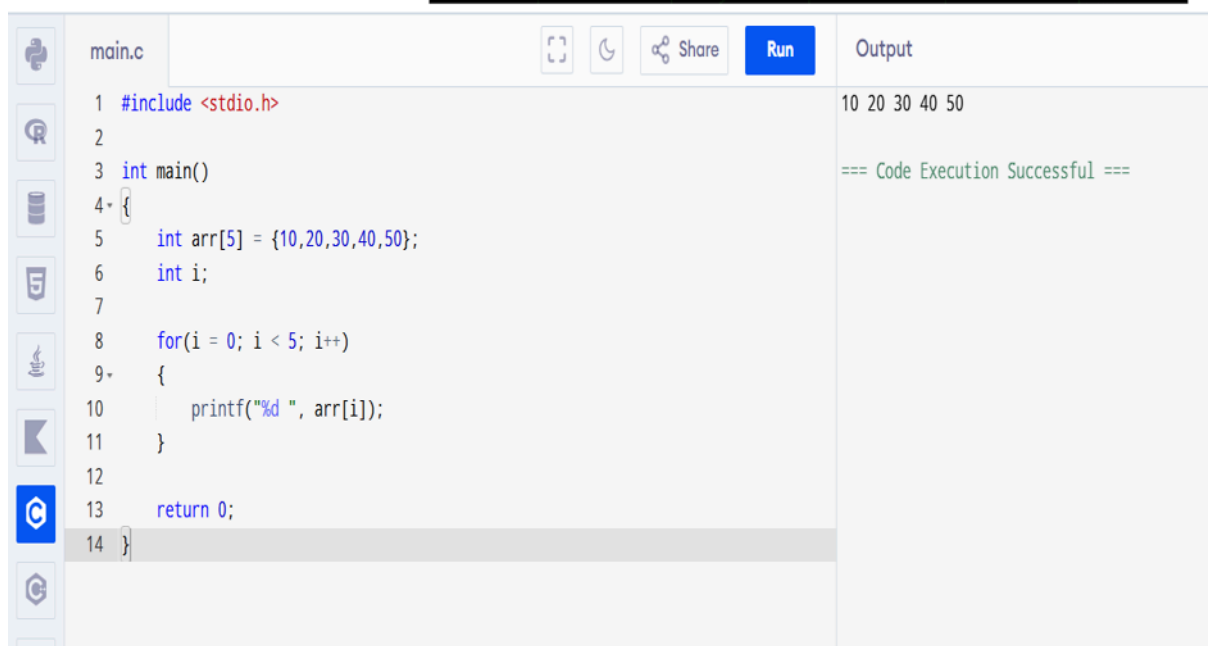
        printf("%d ",arr[i]);

    return 0;

}
```

Expected output:

10 0 30 40 50



The screenshot shows a C code editor with a file named 'main.c'. The code is as follows:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int arr[5] = {10,20,30,40,50};
6     int i;
7
8     for(i = 0; i < 5; i++)
9     {
10         printf("%d ", arr[i]);
11     }
12
13     return 0;
14 }
```

The code has a bug: the loop condition is `i < 5`, which means the loop will not execute for `i = 5`. However, the array `arr` has 5 elements, indexed from 0 to 4. The output of the program is `10 20 30 40 50`, which is the correct output for the given code. The output is displayed in the 'Output' panel on the right, along with the message `=== Code Execution Successful ===`.

3. Identify an errors in the following snippet

```
int binarySearch(int arr[], int n, int key)
{
    int low = 0, high = n - 1;

    while(low < high)

    { int mid = (low + high) / 2;

    if(arr[mid] == key) return mid;

    else if(arr[mid] < key)

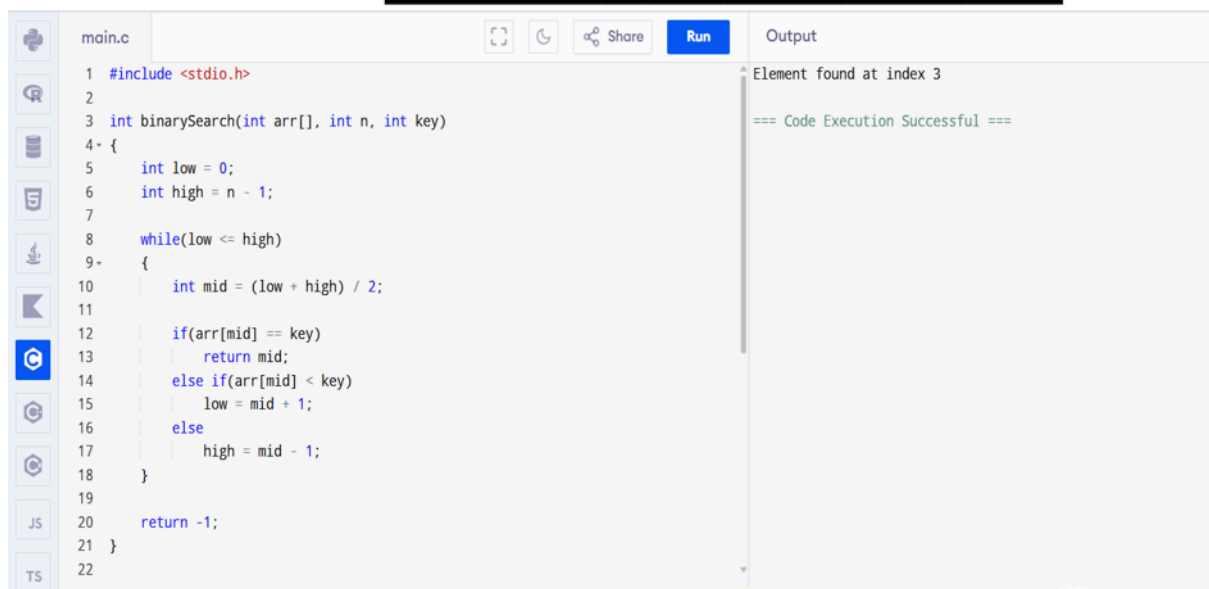
    low = mid;

    else high = mid;

    }

    return -1;

}
```



The screenshot shows a C++ IDE with a file named 'main.c'. The code is a binary search function. The output window shows 'Element found at index 3' and '=== Code Execution Successful ==='.

```
main.c
1 #include <stdio.h>
2
3 int binarySearch(int arr[], int n, int key)
4 {
5     int low = 0;
6     int high = n - 1;
7
8     while(low <= high)
9     {
10         int mid = (low + high) / 2;
11
12         if(arr[mid] == key)
13             return mid;
14         else if(arr[mid] < key)
15             low = mid + 1;
16         else
17             high = mid - 1;
18     }
19
20     return -1;
21 }
22
```

Output

Element found at index 3

=== Code Execution Successful ===

4. Buggy Code in Stack Push Operation

```
#include <stdio.h>

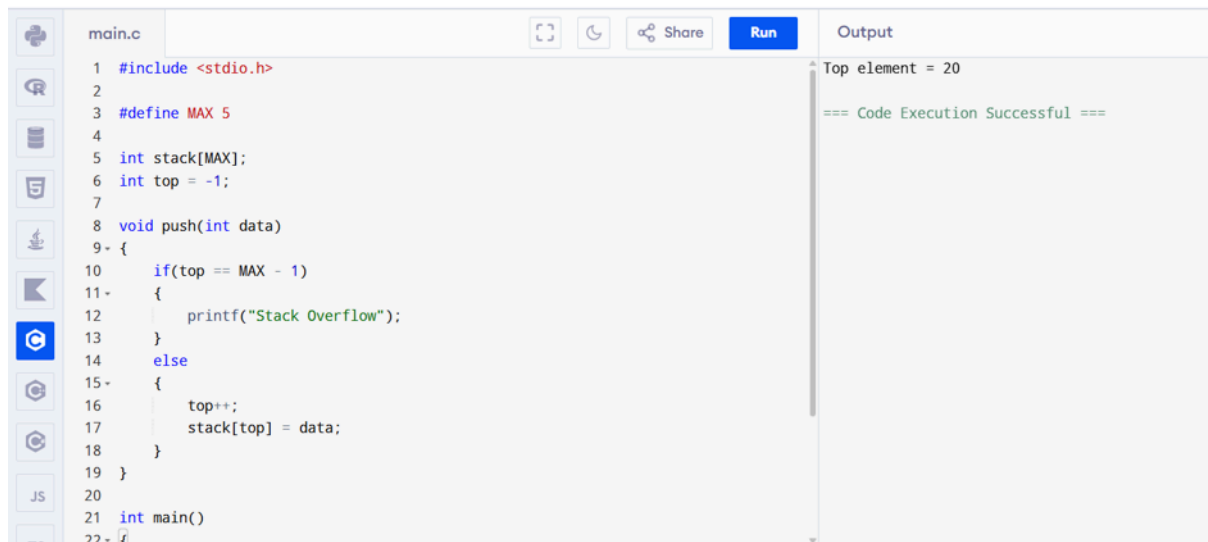
#define MAX 5

int stack[MAX];
int top = -1;

void push(int data) {
    if (top == MAX - 1)
        printf("Stack Overflow");
    else
        stack[top++] = data;
}

int main() {
    push(10);
    push(20);

    printf("%d", stack[top]);
}
```



The screenshot shows a code editor with a file named 'main.c'. The code is as follows:

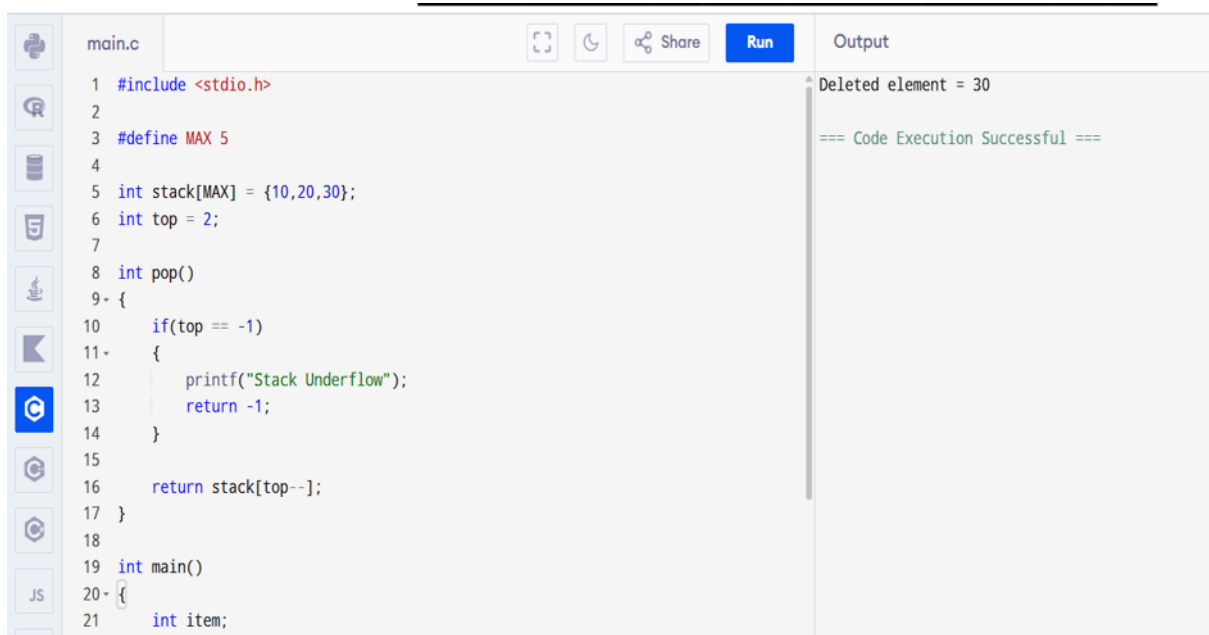
```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int stack[MAX];
6 int top = -1;
7
8 void push(int data)
9 {
10     if(top == MAX - 1)
11     {
12         printf("Stack Overflow");
13     }
14     else
15     {
16         top++;
17         stack[top] = data;
18     }
19 }
20
21 int main()
22 {
```

The output window on the right shows the following text:

```
Top element = 20
=== Code Execution Successful ===
```

5. Buggy Code in Stack POP Operation

```
#include <stdio.h>
#define MAX 5
int stack[MAX];
int top = 2;
int pop() {
    if (top == -1)
        return -1;
    return stack[++top];
}
```



The screenshot shows a C code editor with a file named `main.c`. The code defines a stack with a maximum size of 5 and an initial `top` index of 2. The `pop()` function is implemented with a bug: it checks for underflow (`top == -1`) but then increments `top` before returning the element at `stack[top]`. The `main` function is partially visible, starting with `int main()` and `{`. The output panel on the right shows the result of running the code: "Deleted element = 30" and "=== Code Execution Successful ===".

```
main.c
1  #include <stdio.h>
2
3  #define MAX 5
4
5  int stack[MAX] = {10,20,30};
6  int top = 2;
7
8  int pop()
9  {
10     if(top == -1)
11     {
12         printf("Stack Underflow");
13         return -1;
14     }
15
16     return stack[top--];
17 }
18
19 int main()
20 {
21     int item;
```

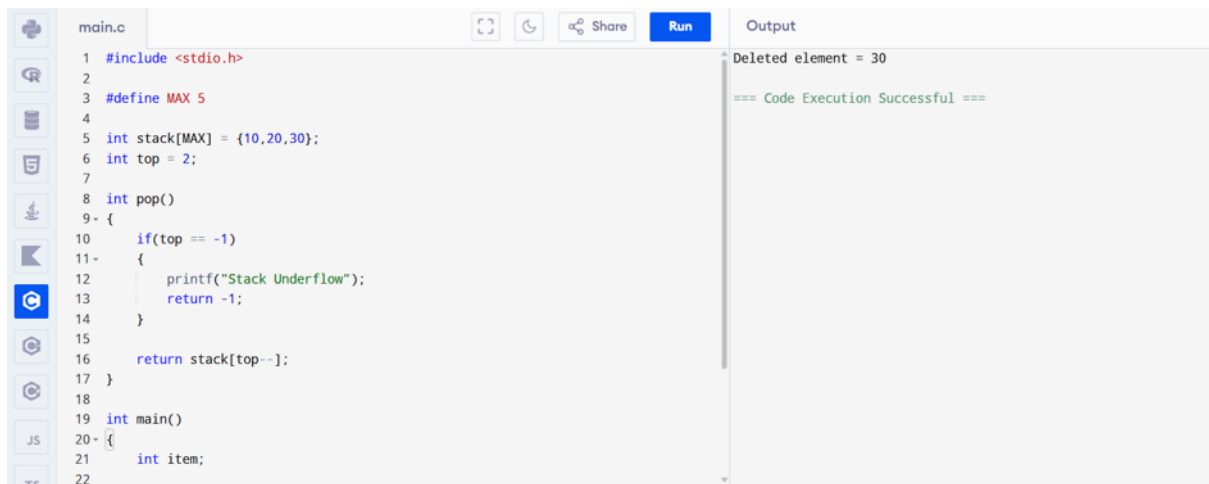
Output

Deleted element = 30

=== Code Execution Successful ===

6. Buggy Code in underflow operation

```
int pop() {  
    if(top == 0) {  
        printf("Underflow");  
        return -1;  
    }  
    return stack[top--];  
}
```



The screenshot shows a C code editor with a file named `main.c`. The code defines a stack with a maximum size of 5 and initializes it with the values {10, 20, 30}. The `pop()` function is implemented with a bug: it checks if `top` is equal to -1 instead of 0. Since `top` is initialized to 2, the condition `if(top == -1)` is false, and the function proceeds to return `stack[top--]`, which is 30. The output window shows "Deleted element = 30" and "=== Code Execution Successful ===".

```
main.c  
1 #include <stdio.h>  
2  
3 #define MAX 5  
4  
5 int stack[MAX] = {10,20,30};  
6 int top = 2;  
7  
8 int pop()  
9 {  
10     if(top == -1)  
11     {  
12         printf("Stack Underflow");  
13         return -1;  
14     }  
15  
16     return stack[top--];  
17 }  
18  
19 int main()  
20 {  
21     int item;  
22 }
```

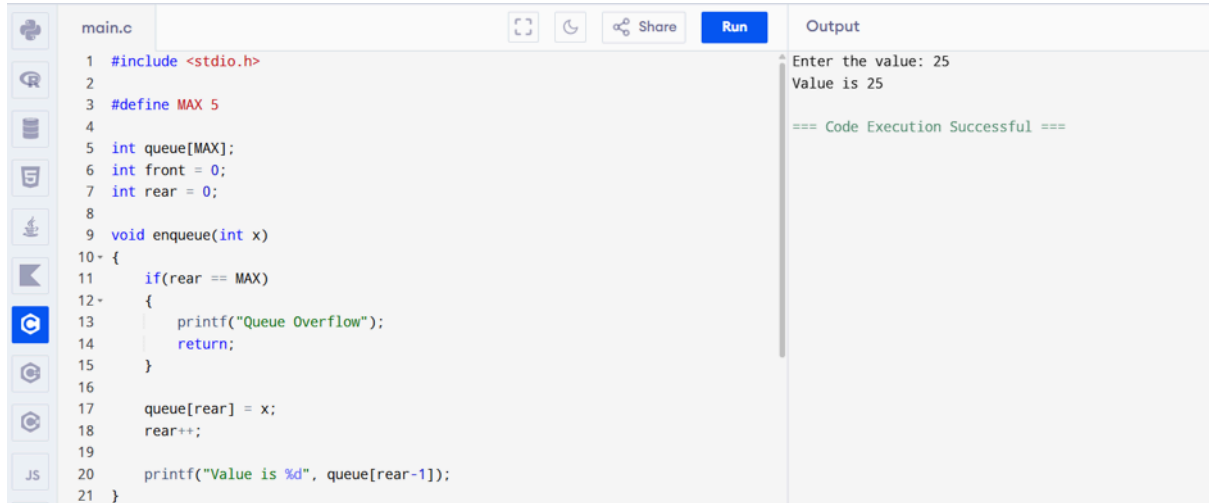
Output
Deleted element = 30
=== Code Execution Successful ===

7. Find the issue in the following code snippet.

```
#include<stdio.h>

#define MAX 5

void main()
{
int queue[MAX],x;
int front = 0, rear = 0;
printf("Enter the value");
scanf("%d",&x);
void enqueue(int x) {
    if(rear == MAX) {
        printf("Overflow");
        return;
    }
    else{
        queue[rear++] = x;
        printf("Value is %d", queue[rear]);
    }
}}
```



The screenshot shows a code editor with a file named 'main.c'. The code is as follows:

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int queue[MAX];
6 int front = 0;
7 int rear = 0;
8
9 void enqueue(int x)
10 {
11     if(rear == MAX)
12     {
13         printf("Queue Overflow");
14         return;
15     }
16
17     queue[rear] = x;
18     rear++;
19
20     printf("Value is %d", queue[rear-1]);
21 }
```

The editor has a toolbar with icons for file operations, a 'Share' button, and a 'Run' button. The output window on the right shows the following text:

```
Enter the value: 25
Value is 25
=== Code Execution Successful ===
```

The code contains a logical error. In the `enqueue` function, the `if` condition `if(rear == MAX)` checks for overflow, but the `printf` statement `printf("Value is %d", queue[rear-1]);` is executed regardless of whether the overflow check passed or failed. This means that if the queue is full, the program will still attempt to print the value at `queue[rear-1]`, which is the last element added, but it will not print the value of `x` that was passed to the function. The output shows that the value 25 was printed, which is the value of `x`, indicating that the overflow check was not working as intended.

8. Buggy code in node creation

```
#include <stdio.h>
#include <stdlib.h>

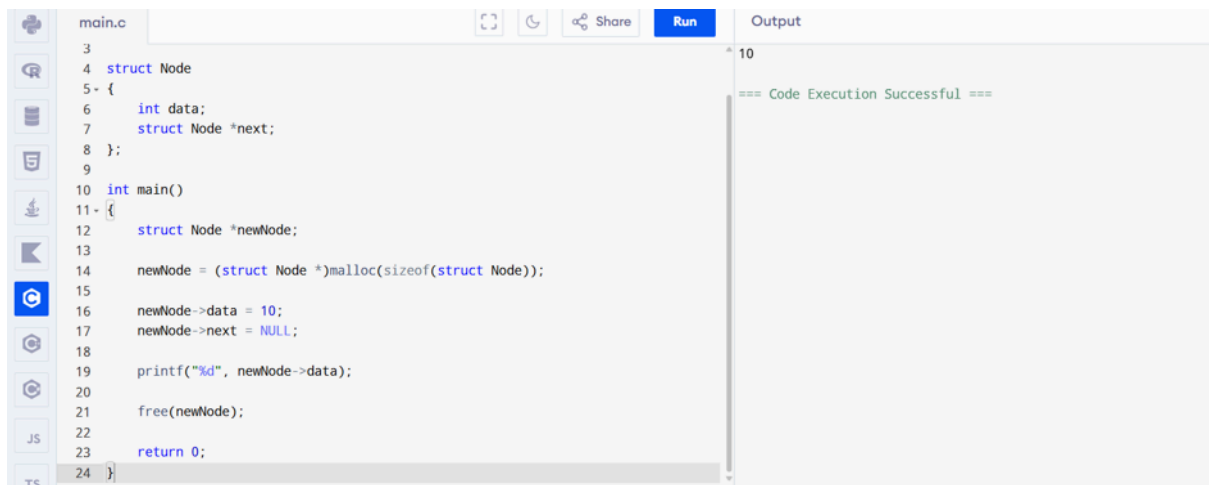
struct Node {
    int data;
    struct Node *next;
};

int main() {
    struct Node *newNode;

    newNode->data = 10;
    newNode->next = NULL;

    printf("%d", newNode->data);

    return 0;
}
```



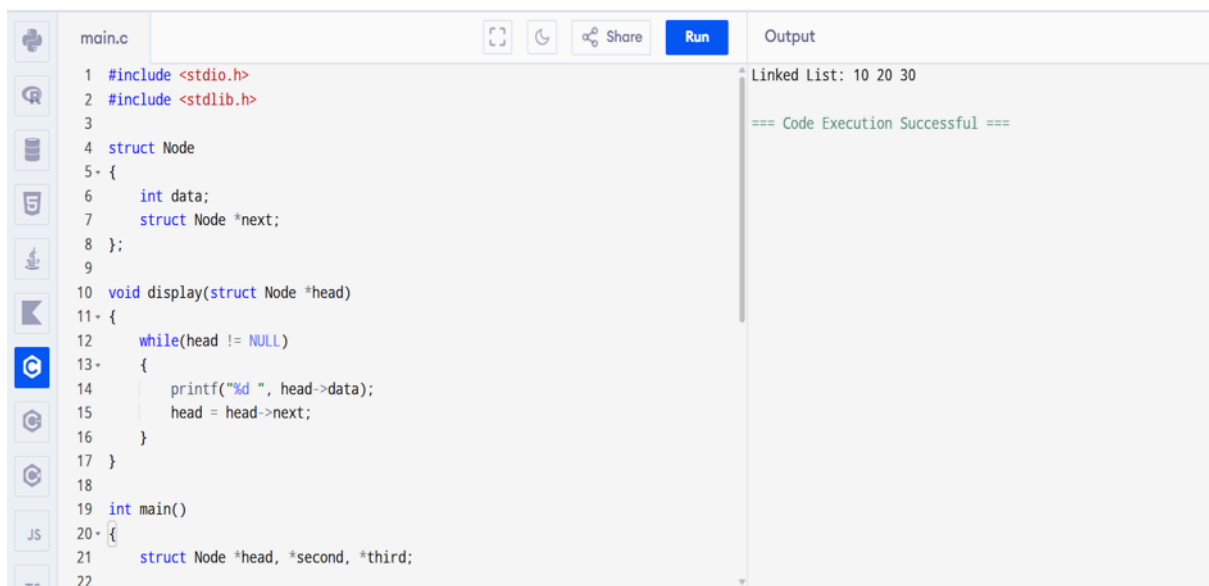
The screenshot shows a code editor with a file named `main.c`. The code is as follows:

```
3
4 struct Node
5 {
6     int data;
7     struct Node *next;
8 };
9
10 int main()
11 {
12     struct Node *newNode;
13
14     newNode = (struct Node *)malloc(sizeof(struct Node));
15
16     newNode->data = 10;
17     newNode->next = NULL;
18
19     printf("%d", newNode->data);
20
21     free(newNode);
22
23     return 0;
24 }
```

The editor has a toolbar with icons for undo, redo, share, and a blue **Run** button. To the right of the code editor is an **Output** panel. It displays the number `10` on the first line and `=== Code Execution Successful ===` on the second line.

9. Include missing statements and display the list of elements in linkedlist

```
void display(struct Node *head)
{
while(head != NULL);
{
printf("%d ", head->data);
head = head->next;
}}
```



The screenshot shows a C code editor with a file named `main.c`. The code defines a `struct Node` with an `int data` field and a `struct Node *next` pointer. It implements a `display` function that traverses the linked list and prints the data of each node. The `main` function initializes three nodes with values 10, 20, and 30, and calls the `display` function. The output window shows the result: "Linked List: 10 20 30" and "=== Code Execution Successful ===".

```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int data;
7     struct Node *next;
8 };
9
10 void display(struct Node *head)
11 {
12     while(head != NULL)
13     {
14         printf("%d ", head->data);
15         head = head->next;
16     }
17 }
18
19 int main()
20 {
21     struct Node *head, *second, *third;
22 }
```

Output

Linked List: 10 20 30

=== Code Execution Successful ===

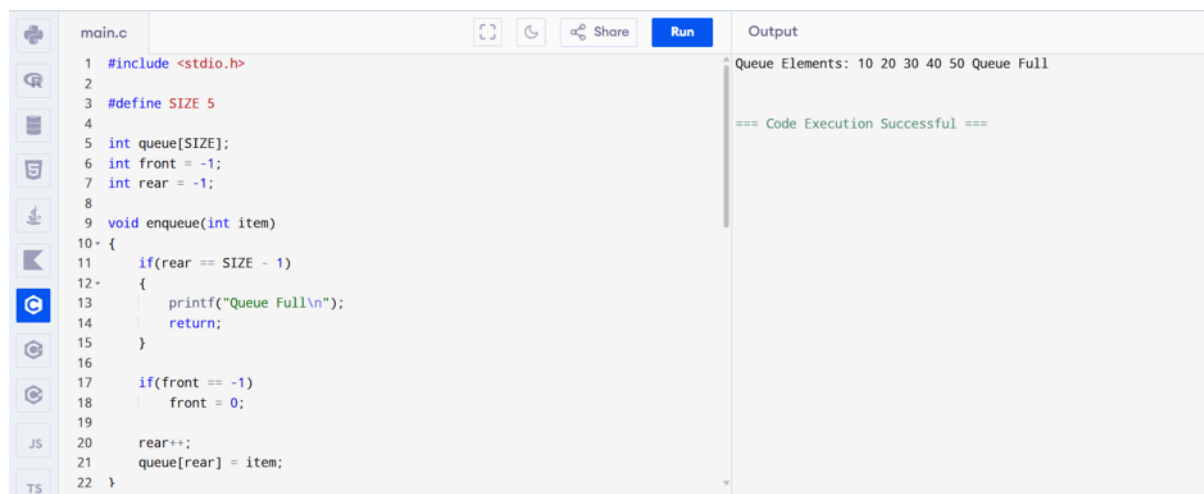
10. Bug in Queue Full Condition-Identify and fix the errors.

```
#define SIZE 5
int queue[SIZE];
int front = -1, rear = -1;

void enqueue(int item)
{
    if(rear == SIZE-1)
    {
        printf("Queue Full");
        return;
    }

    if(front == -1)
        front = 0;

    rear = (rear + 1) % SIZE;
    queue[rear] = item;
}
```



The screenshot shows a C code editor with a file named 'main.c'. The code implements a queue with a size of 5. It defines an array 'queue' and variables 'front' and 'rear', both initialized to -1. The 'enqueue' function checks if 'rear' is equal to 'SIZE - 1' (4) to determine if the queue is full. If so, it prints 'Queue Full' and returns. Otherwise, it sets 'front' to 0 if it's -1, increments 'rear' modulo 'SIZE', and adds the item to the queue. The code is executed, and the output shows 'Queue Elements: 10 20 30 40 50 Queue Full' followed by '=== Code Execution Successful ==='.

```
main.c
1  #include <stdio.h>
2
3  #define SIZE 5
4
5  int queue[SIZE];
6  int front = -1;
7  int rear = -1;
8
9  void enqueue(int item)
10 {
11     if(rear == SIZE - 1)
12     {
13         printf("Queue Full\n");
14         return;
15     }
16
17     if(front == -1)
18         front = 0;
19
20     rear++;
21     queue[rear] = item;
22 }
```

Output

Queue Elements: 10 20 30 40 50 Queue Full

=== Code Execution Successful ===

Code of Conduct:

I Sarah Sharmeen. R (192571073) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**RUBRICS FOR CALCULATION:**

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

